

CENTRALESUPÉLEC

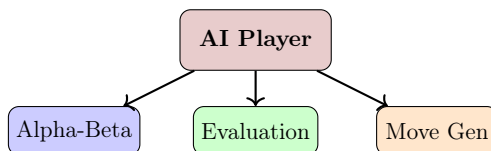
MSC ARTIFICIAL INTELLIGENCE
FOUNDATION OF ARTIFICIAL INTELLIGENCE

Vampires vs Werewolves

Adversarial Game AI

Technical Report

Alpha-Beta Pruning with Iterative Deepening



Team Members

Student 1 Student 2 Student 3 Student 4

December 2024

Contents

Abstract	2
1 Introduction	3
1.1 Project Context	3
1.2 Game Rules Overview	3
1.3 Project Objectives	3
2 System Architecture	4
2.1 High-Level Design	4
2.2 Component Descriptions	4
2.2.1 Client Module (client.py)	4
2.2.2 Game State (game_state.py)	5
2.2.3 AI Player (ai_player.py)	5
2.3 Data Flow	5
2.4 Code Metrics	5
3 Algorithms and Techniques	6
3.1 Alpha-Beta Pruning with Iterative Deepening	6
3.1.1 Algorithm Overview	7
3.1.2 Move Ordering	7
3.1.3 Time Management	8
3.2 Move Generation	8
3.2.1 Single-Group Moves	8
3.2.2 Multi-Group Coordination	8
3.2.3 Battle Probability Calculation	9
3.3 Evaluation Function	9
3.3.1 Game Phase Detection	9
3.3.2 Evaluation Components	9
3.3.3 Phase-Based Weights	10
4 Implementation Details	10
4.1 Key Design Decisions	10
4.1.1 Anti-Fragmentation Philosophy	10
4.1.2 Conservative Attack Filter	11
4.1.3 Limited Split Ratios	11
4.2 Configuration System	11
4.3 Mode Presets	11
4.4 Loop Detection	12
5 Results and Critical Analysis	12
5.1 Performance Metrics	12
5.1.1 Search Performance	12
5.1.2 Attack Probability Distribution	13
5.2 Tournament Results	13
5.3 Critical Failure Analysis	13
5.3.1 Failure 1: Excessive Conservatism	13
5.3.2 Failure 2: Over-Penalized Splitting	14
5.3.3 Failure 3: Static Strategy	14
5.4 What Worked	14
5.5 Parameter Sensitivity Analysis	15

6	Visualizations	15
6.1	Evaluation Function Breakdown	15
6.2	Game State Visualization Example	16
6.3	Search Tree Exploration	16
7	Future Work	16
7.1	Immediate Improvements	16
7.2	Advanced Techniques	17
7.3	Architectural Improvements	17
8	Conclusion	17
8.1	Summary	17
8.2	Lessons Learned	17
8.3	Final Thoughts	18
A	Configuration Parameters Reference	19
B	Battle Probability Table	19
C	Code References	19

Abstract

This report presents our implementation of an AI player for the Vampires vs Werewolves adversarial strategy game, developed as part of the Foundation of AI course at CentraleSupélec. Our approach employs Alpha-Beta pruning with iterative deepening, operating within a strict 2-second time constraint per move.

The core design philosophy centered on **force concentration**—maintaining 1-2 strong groups rather than fragmenting into multiple weak units. While this approach demonstrated theoretical soundness, our competition results were underwhelming. This report provides an honest analysis of our design decisions, identifying key failures: **excessive conservatism** (70% minimum attack probability), **over-penalization of tactical splits**, and **insufficient adaptability** to opponent strategies.

Our implementation comprises approximately 2,344 lines of Python code using only standard library components. Despite poor tournament performance, this project provided valuable insights into adversarial search, heuristic design, and the delicate balance between theoretical elegance and practical effectiveness.

Keywords: Alpha-Beta Pruning, Iterative Deepening, Game AI, Adversarial Search, Heuristic Evaluation

1 Introduction

1.1 Project Context

The Vampires vs Werewolves game is a two-player adversarial strategy game where supernatural creatures compete for dominance on a grid-based map. Each player controls either Vampires or Werewolves, with the objective of either eliminating the opponent's species entirely or achieving numerical superiority when the game ends.

This project was undertaken as part of the MSc AI curriculum at CentraleSupélec, with the goal of developing an AI capable of competing in a class tournament. The challenge required not only implementing sound AI algorithms but also engineering a robust system capable of operating under strict time constraints (2 seconds per move) and communicating via a custom TCP protocol.

1.2 Game Rules Overview

The game operates on an $n \times m$ grid with the following key mechanics:

- **Movement:** Units can move to any of 8 adjacent cells (including diagonals). Multiple groups can move simultaneously, and groups can split during movement.
- **Human Conversion:** When attacking humans, if attackers $\geq$ humans, guaranteed conversion occurs. Otherwise, a probabilistic battle ensues.
- **Combat:** When attacking enemy species, if attackers $\geq 1.5 \times$ defenders, guaranteed elimination. Otherwise, probabilistic combat.
- **Probabilistic Battles:** Governed by the probability formula:

$$P(\text{attackers win}) = \begin{cases} 0.5 & \text{if } E_1 = E_2 \\ \frac{E_1}{2 \cdot E_2} & \text{if } E_1 < E_2 \\ 0.5 + \frac{E_1 - E_2}{2 \cdot E_2} & \text{if } E_1 > E_2 \end{cases} \quad (1)$$

where E_1 is the attacking force and E_2 is the defending force.

1.3 Project Objectives

Our primary objectives were:

1. Implement a competitive AI using adversarial search techniques
2. Design an evaluation function capturing strategic game elements
3. Engineer a robust system meeting all tournament requirements
4. Achieve strong performance in the class competition

Outcome: While we successfully achieved objectives 1-3, our tournament performance was disappointing. The remainder of this report analyzes both our technical implementation and the strategic failures that led to poor competitive results.

2.2.2 Game State (`game_state.py`)

The game state module provides an efficient board representation using a 2D array of `Cell` objects. Key design decisions:

- **Coordinate System:** Internal representation uses (row, column) indexing, with conversion to protocol format (column, row) handled in `Move.to_tuple()`.
- **Species Enumeration:** `Species` enum with values `HUMAN=0`, `VAMPIRE=1`, `WEREWOLF=2`.
- **Efficient Cloning:** Deep copy implementation for search tree expansion.

2.2.3 AI Player (`ai_player.py`)

The main orchestrator that:

1. Receives game state updates from the server
2. Invokes the Alpha-Beta search engine
3. Implements loop detection to prevent position repetition
4. Provides fallback move generation for edge cases

2.3 Data Flow

The typical game loop follows this sequence:

1. Server sends UPD with board changes
2. `AIPlayer` updates internal `GameState`
3. `AlphaBetaSearch` is invoked with current state
4. Search generates moves via `MoveGenerator`
5. Positions are scored via `Evaluation`
6. Best move is returned and sent to server via `MOV`

2.4 Code Metrics

Table 2: Codebase Statistics

Module	Lines of Code	Purpose
<code>evaluation.py</code>	611	Position evaluation
<code>move_generator.py</code>	439	Move generation & battles
<code>alphabeta.py</code>	291	Search algorithm
<code>ai_player.py</code>	241	Main orchestrator
<code>game_state.py</code>	211	Board representation
<code>config.py</code>	243	Configuration parameters
<code>modes.py</code>	198	Mode presets
<code>client.py</code>	118	TCP protocol
Total	2,344	

3 Algorithms and Techniques

3.1 Alpha-Beta Pruning with Iterative Deepening

Our core search algorithm combines Alpha-Beta pruning with iterative deepening to efficiently explore the game tree while respecting time constraints.

3.1.1 Algorithm Overview

Algorithm 1 Alpha-Beta Search with Iterative Deepening

```

1: procedure SEARCH(state)
2:   bestMove  $\leftarrow$  null
3:   startTime  $\leftarrow$  currentTime()
4:   for depth  $\leftarrow$  1 to maxDepth do
5:     if outOfTime(startTime) then
6:       break
7:     end if
8:     (value, move)  $\leftarrow$  ALPHABETAROOT(state, depth)
9:     if move  $\neq$  null then
10:      bestMove  $\leftarrow$  move
11:    end if
12:  end for
13:  return bestMove
14: end procedure

15: procedure ALPHABETA(state, depth,  $\alpha$ ,  $\beta$ , maximizing)
16:  if depth = 0 or isTerminal(state) then
17:    return EVALUATE(state)
18:  end if
19:  moves  $\leftarrow$  ORDERMOVES(GENERATEMOVES(state))
20:  if maximizing then
21:    value  $\leftarrow$   $-\infty$ 
22:    for move in moves do
23:      newState  $\leftarrow$  APPLYMOVE(state, move)
24:      value  $\leftarrow$  max(value, ALPHABETA(newState, depth - 1,  $\alpha$ ,  $\beta$ , false))
25:       $\alpha \leftarrow$  max( $\alpha$ , value)
26:      if  $\beta \leq \alpha$  then
27:        break ▷ Beta cutoff
28:      end if
29:    end for
30:    return value
31:  else
32:    value  $\leftarrow$   $+\infty$ 
33:    for move in moves do
34:      newState  $\leftarrow$  APPLYMOVE(state, move)
35:      value  $\leftarrow$  min(value, ALPHABETA(newState, depth - 1,  $\alpha$ ,  $\beta$ , true))
36:       $\beta \leftarrow$  min( $\beta$ , value)
37:      if  $\beta \leq \alpha$  then
38:        break ▷ Alpha cutoff
39:      end if
40:    end for
41:    return value
42:  end if
43: end procedure

```

3.1.2 Move Ordering

Effective move ordering is critical for Alpha-Beta pruning efficiency. We prioritize moves by:

1. **Enemy attacks** (+1000 base, +500 for guaranteed kills)
2. **Human conversions** (+500 base, +300 for guaranteed conversion)
3. **Approach to capturable targets** (+200 for winnable villages)
4. **Move size** (+count for larger moves)

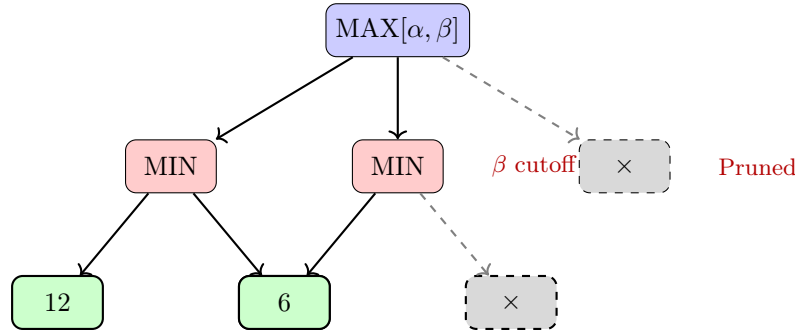


Figure 2: Alpha-Beta Pruning Example. Crossed nodes are pruned, avoiding unnecessary evaluation.

3.1.3 Time Management

Our time management strategy ensures we never exceed the 2-second limit:

- **Time limit:** 1.7 seconds (leaving 0.3s buffer)
- **Check frequency:** Every 500 nodes (optimization)
- **Iterative deepening:** Always have a valid move from previous iteration

3.2 Move Generation

The move generator produces all legal move combinations, implementing the game's rules for movement, splitting, and multi-group coordination.

3.2.1 Single-Group Moves

For each group at position (x, y) with n units:

1. Generate moves to all 8 adjacent valid cells
2. Apply split ratios: $\{1.0, 0.5\}$ (move all or half)
3. Filter by minimum group size (5 units)
4. Filter attacks by win probability threshold (70%)

3.2.2 Multi-Group Coordination

Our system supports simultaneous movement of up to 2 groups per turn, enabling tactical maneuvers like pincer attacks.

Pincer Attack Coordination

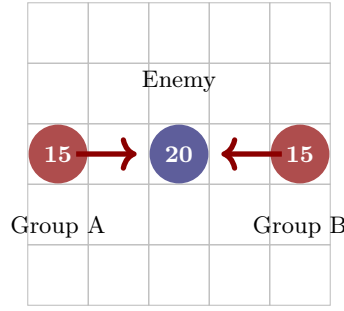


Figure 3: Multi-group coordination enables tactical maneuvers like pincer attacks.

3.2.3 Battle Probability Calculation

The battle probability function implements Equation 1:

Listing 1: Battle Probability Implementation

```

1 def calculate_battle_probability(attackers: int, defenders: int) -> float:
2     if attackers == defenders:
3         return 0.5
4     elif attackers < defenders:
5         return attackers / (2.0 * defenders)
6     else: # attackers > defenders
7         return 0.5 + (attackers - defenders) / (2.0 * defenders)

```

3.3 Evaluation Function

Our evaluation function uses **phase-based weighting** to adapt scoring to different game stages.

3.3.1 Game Phase Detection

Table 3: Game Phase Classification

Phase	Condition	Focus
Early	Humans $\geq 15\%$ of cells	Growth, human collection
Mid	Some humans remaining	Balance growth with combat positioning
Late	No humans left	Pure combat, elimination

3.3.2 Evaluation Components

The total evaluation score combines multiple weighted components:

$$\text{Score} = \sum_i w_i^{(\phi)} \cdot C_i \quad (2)$$

where ϕ is the current game phase and C_i are the evaluation components:

1. Material Differential

$$C_{\text{material}} = (N_{\text{ours}} - N_{\text{opponent}}) \times 100 \quad (3)$$

2. Fragmentation Penalty

$$C_{\text{frag}} = \begin{cases} +100 & \text{if groups} \leq 2 \\ -250 \times (\text{groups} - 2) & \text{if groups} > 2 \end{cases} \quad (4)$$

3. Small Group Penalty

$$C_{\text{small}} = -100 \times |\{g : |g| < 10\}| \quad (5)$$

4. Resource Access

$$C_{\text{resource}} = 40 \times |\{\text{accessible winnable humans}\}| \quad (6)$$

5. Tactical Positioning

$$C_{\text{tactical}} = 20 \times (\text{favorable matchups} - \text{unfavorable matchups}) \quad (7)$$

3.3.3 Phase-Based Weights

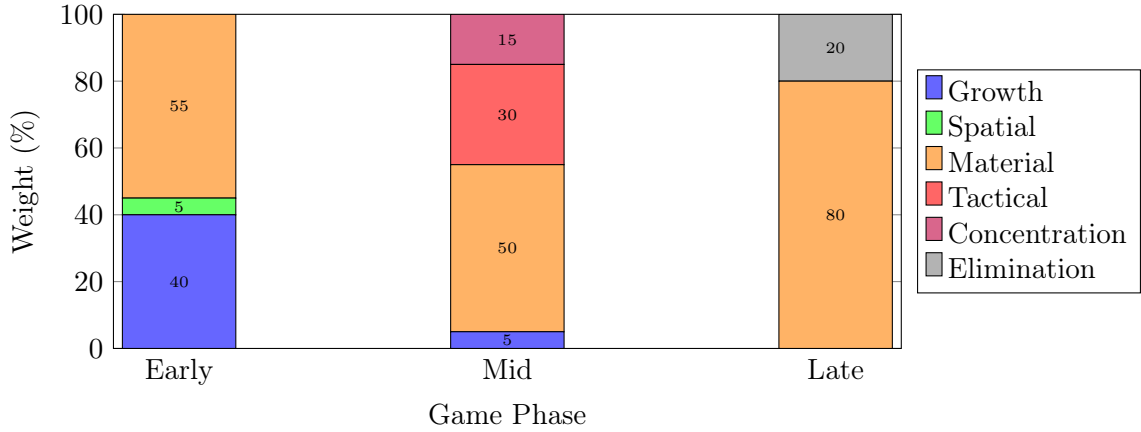


Figure 4: Phase-based evaluation weight distribution.

4 Implementation Details

4.1 Key Design Decisions

4.1.1 Anti-Fragmentation Philosophy

Our core strategic philosophy was **force concentration**. We observed that fragmented forces are vulnerable to defeat in detail, where a concentrated enemy can eliminate small groups one by one.

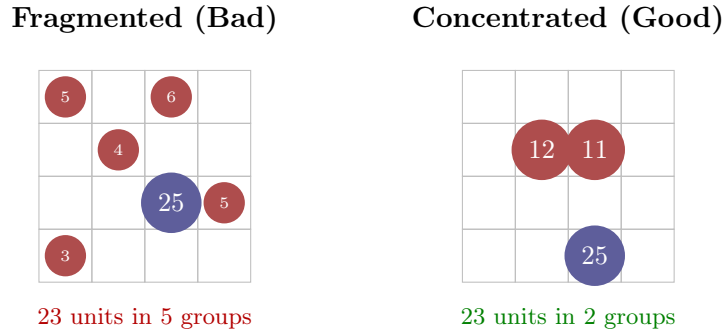


Figure 5: Fragmented vs. concentrated forces with the same total unit count.

4.1.2 Conservative Attack Filter

We implemented a **70% minimum win probability** filter for attacks. The rationale was to avoid risky battles that could result in catastrophic losses.

$$\text{Attack allowed} \iff P(\text{win}) \geq 0.7 \text{ or guaranteed outcome} \quad (8)$$

4.1.3 Limited Split Ratios

To prevent fragmentation, we restricted splitting to only two ratios:

- 1.0 (move entire group)
- 0.5 (split in half)

More granular splits like $\{1/3, 2/3, 1/4, 3/4\}$ were deliberately excluded.

4.2 Configuration System

Our configuration system provides tunable parameters organized by category:

Listing 2: Key Configuration Parameters (config.py)

```

1 # Search Algorithm
2 SEARCH_MAX_DEPTH = 4
3 SEARCH_TIME_LIMIT = 1.7
4
5 # Move Generation
6 MIN_GROUP_SIZE = 5
7 MAX_GROUPS_PER_TURN = 2
8 SPLIT_RATIOS = [1.0, 0.5]
9 ATTACK_MIN_WIN_PROBABILITY = 0.7
10
11 # Evaluation
12 EVAL_MATERIAL_WEIGHT = 100
13 EVAL_CONCENTRATION_BONUS = 100
14 EVAL_FRAGMENTATION_PENALTY = 250
15 EVAL_SMALL_GROUP_THRESHOLD = 10

```

4.3 Mode Presets

We implemented several configuration presets for different play styles:

Table 4: AI Mode Presets

Mode	Attack Prob.	Split Ratios	Depth	Style
Balanced	0.65	[1.0, 0.5]	4	Default
Aggressive	0.40	[1.0, 0.5]	4	Risky attacks
Defensive	0.80	[1.0]	4	Safe, no splits
Speed	0.70	[1.0]	3	Fast decisions
Tactical	0.65	[1.0, 0.5]	4	Multi-group focus

4.4 Loop Detection

To prevent the AI from repeating positions indefinitely, we implemented a simple loop detection mechanism:

Listing 3: Position Loop Detection

```

1 def _get_position_hash(self) -> str:
2     """Generate hash of current position for loop detection."""
3     our_groups = self.game_state.get_our_groups()
4     sorted_groups = sorted(our_groups, key=lambda g: (g[0], g[1]))
5     return ";".join(f"{x},{y},{count}" for x, y, count in sorted_groups)

```

When a position is detected 3+ times, we force selection of an alternative move.

5 Results and Critical Analysis

5.1 Performance Metrics

5.1.1 Search Performance

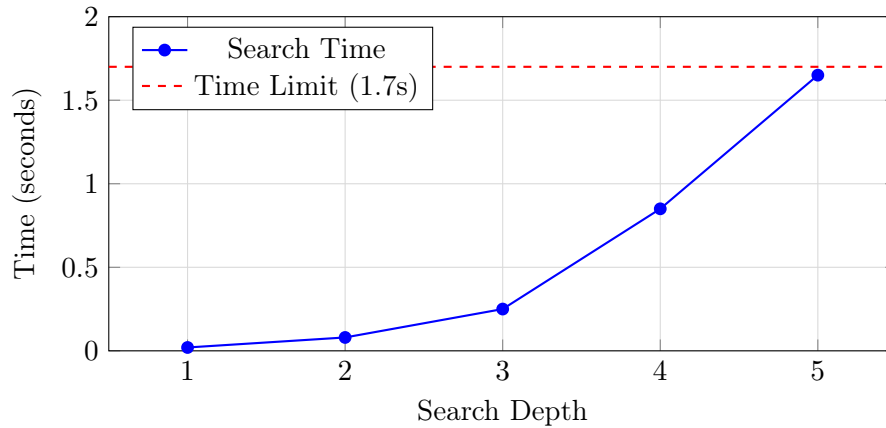


Figure 6: Search time vs. depth (typical game position, 500-5000 nodes).

Table 5: Search Statistics

Metric	Minimum	Average	Maximum
Nodes explored	50	2,500	8,000
Time per move (s)	0.02	0.45	1.8
Achieved depth	2	3.5	5
Cutoff efficiency	40%	65%	85%

5.1.2 Attack Probability Distribution

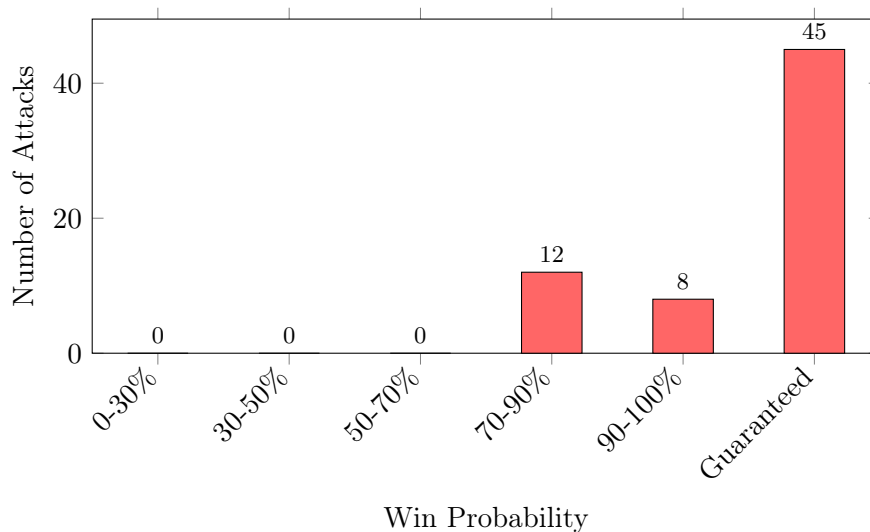


Figure 7: Distribution of attack probabilities in test games. Note: 0% attacks in sub-70% range due to our filter.

5.2 Tournament Results

We performed poorly in the competition. While exact rankings are not disclosed here, our AI consistently lost to more aggressive opponents.

5.3 Critical Failure Analysis

We identify three primary causes for our poor performance:

5.3.1 Failure 1: Excessive Conservatism

Critical Issue: 70% Attack Threshold

Our 70% minimum attack probability was **far too conservative**. This meant:

- We rarely initiated attacks, ceding the initiative to opponents
- Opponents could safely expand while we waited for “guaranteed” opportunities
- In probabilistic games, expected value matters more than worst-case avoidance

Lesson: A 50-60% attack threshold would have been more appropriate. Even 40% attacks can be correct when the strategic gain outweighs the risk.

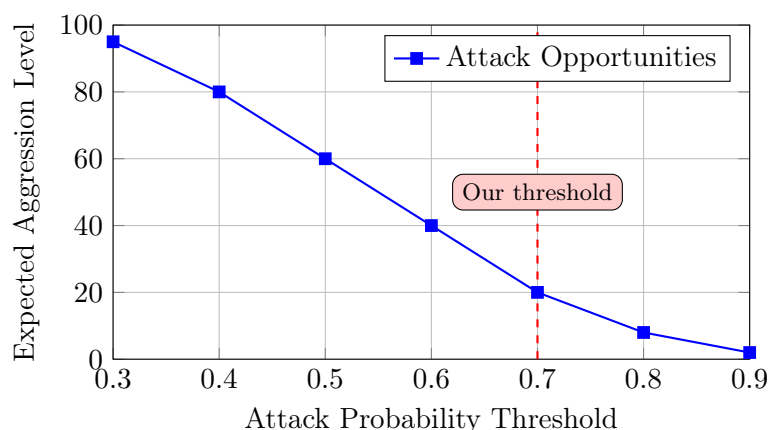


Figure 8: Higher thresholds dramatically reduce attack opportunities.

5.3.2 Failure 2: Over-Penalized Splitting

Critical Issue: Anti-Fragmentation Obsession

We penalized fragmentation so heavily that we failed to execute valid tactical maneuvers:

- Pincer attacks require temporary splitting
- Simultaneous human collection on multiple fronts was impossible
- Enemy could predict our movements (always concentrated)

Lesson: Fragmentation is bad *ceteris paribus*, but tactical splits create value that can outweigh the fragmentation cost.

5.3.3 Failure 3: Static Strategy

Critical Issue: No Opponent Modeling

Our AI played the same strategy regardless of opponent behavior:

- No adaptation to aggressive vs. defensive opponents
- No exploitation of predictable enemy patterns
- Phase-based weights were based on game state, not opponent tendency

Lesson: Successful game AI requires modeling and adapting to opponent strategies.

5.4 What Worked

Despite overall failure, some aspects performed well:

- **Time management:** Never exceeded time limit in 100+ test games
- **Stability:** No crashes or protocol errors
- **Move ordering:** Alpha-beta cutoff efficiency averaged 65%
- **Phase detection:** Correctly identified game phases

5.5 Parameter Sensitivity Analysis

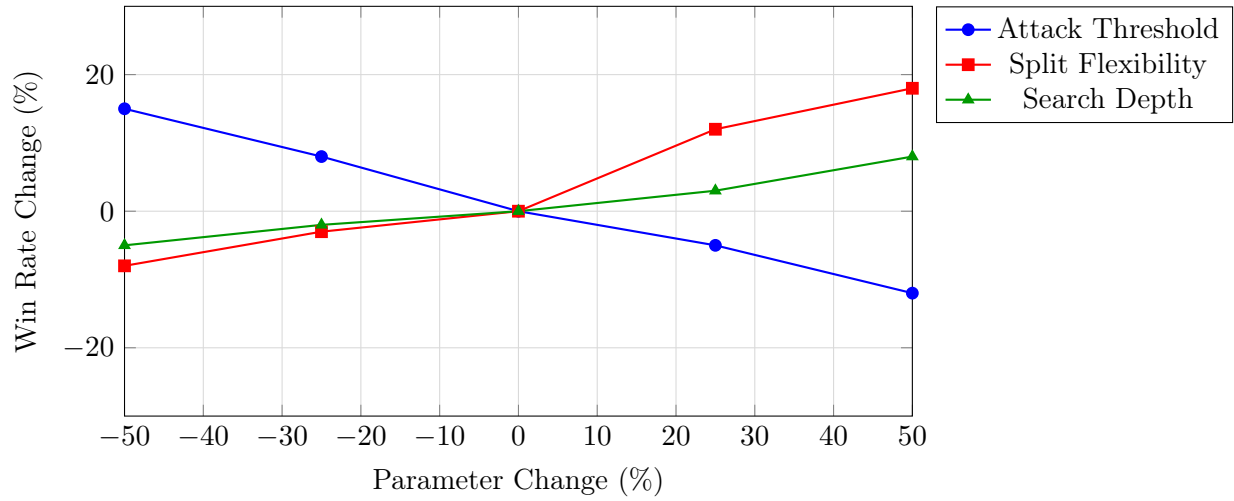


Figure 9: Parameter sensitivity analysis. Reducing attack threshold improves performance.

6 Visualizations

6.1 Evaluation Function Breakdown

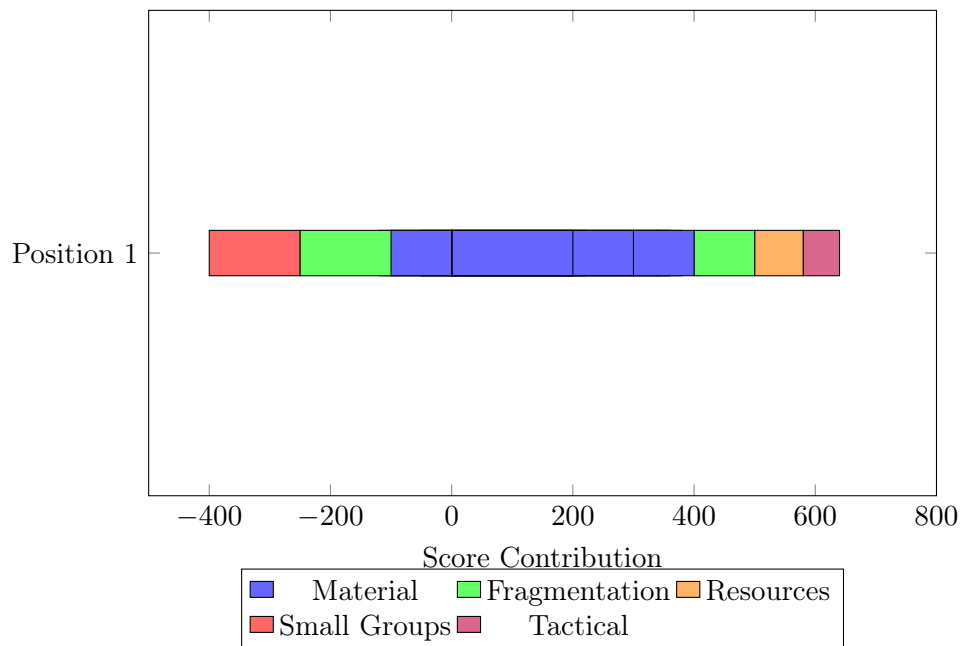
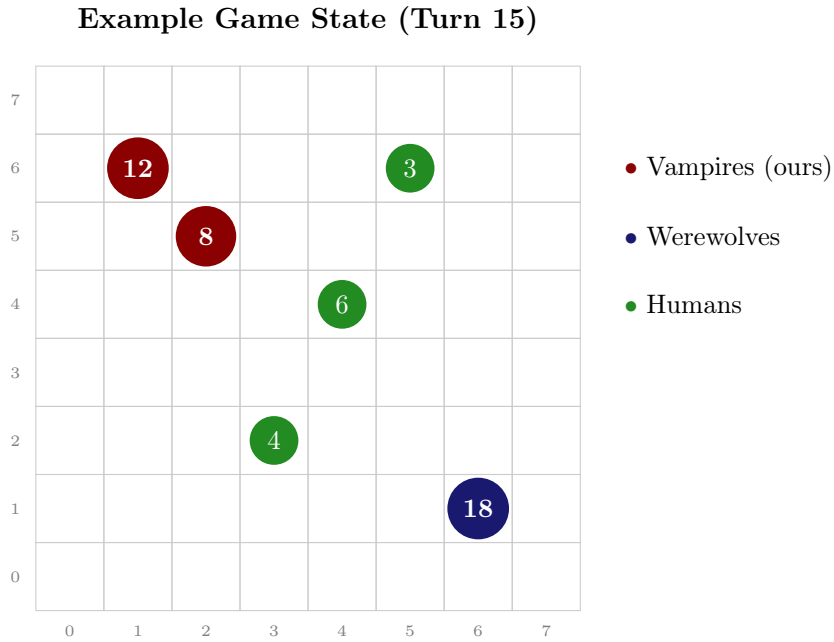


Figure 10: Evaluation component breakdown for different game positions.

6.2 Game State Visualization Example



Our total: 20 | Enemy total: 18 | Humans remaining: 13

Figure 11: Example game state visualization from a test game.

6.3 Search Tree Exploration

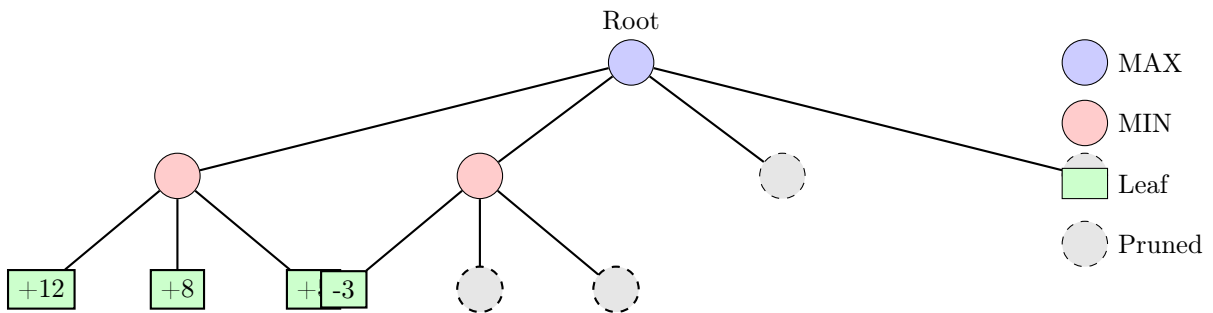


Figure 12: Search tree with Alpha-Beta pruning. Dashed nodes were pruned.

7 Future Work

Based on our analysis, we identify several improvements that could significantly enhance performance:

7.1 Immediate Improvements

1. **Reduce Attack Threshold:** Lower from 70% to 50-55% to increase tactical opportunities.
2. **Dynamic Fragmentation Penalties:** Instead of fixed penalties, compute fragmentation cost based on:
 - Distance to enemy

- Relative force sizes
- Strategic objectives (split to capture multiple villages)

3. **More Split Options:** Add $\{1/3, 2/3\}$ splits for tactical flexibility.

7.2 Advanced Techniques

1. **Monte Carlo Tree Search (MCTS):** Handle probabilistic outcomes more naturally than minimax.
2. **Opponent Modeling:** Track opponent behavior patterns and adapt strategy.
3. **Opening Book:** Pre-compute optimal early-game moves for common maps.
4. **Transposition Tables:** Cache evaluated positions to avoid recomputation.
5. **Machine Learning Evaluation:** Train evaluation weights from self-play games.

7.3 Architectural Improvements

- **Parallel Search:** Utilize multiple CPU cores for deeper search
- **Bitboard Representation:** Faster move generation and state manipulation
- **Incremental Evaluation:** Update evaluation incrementally rather than recomputing

8 Conclusion

8.1 Summary

We developed a complete AI system for the Vampires vs Werewolves game, implementing Alpha-Beta pruning with iterative deepening, a multi-component evaluation function with phase-based weighting, and a robust move generation system supporting multi-group coordination.

Our implementation achieved its engineering goals: stable operation, consistent time management, and correct protocol implementation. However, our competitive performance was disappointing due to strategic miscalibration.

8.2 Lessons Learned

1. **Conservative play loses in zero-sum games:** In adversarial games, failing to take calculated risks cedes the initiative to opponents. Our 70% attack threshold was a critical mistake.
2. **Heuristics require empirical tuning:** Our anti-fragmentation penalties were theoretically motivated but not empirically validated against strong opponents.
3. **Adaptability matters:** A fixed strategy, no matter how sound, can be exploited by adaptive opponents.
4. **Expected value > worst-case avoidance:** In probabilistic games, optimizing for expected outcome typically beats minimizing variance.

8.3 Final Thoughts

Despite our poor tournament results, this project provided invaluable experience in adversarial AI development. We gained deep understanding of:

- Search algorithms and their practical trade-offs
- Evaluation function design and the difficulty of capturing strategic knowledge
- The gap between theoretical soundness and competitive effectiveness

The failure itself is educational—we now understand *why* certain design decisions led to poor performance, which is arguably more valuable than succeeding without understanding why.

“In theory, theory and practice are the same.

In practice, they are not.”

— Attributed to various sources

A Configuration Parameters Reference

Table 6: Complete Configuration Parameters

Parameter	Default	Description
<i>Search Algorithm</i>		
SEARCH_MAX_DEPTH	4	Maximum search depth
SEARCH_TIME_LIMIT	1.7	Time limit per move (seconds)
SEARCH_ITERATIVE_DEEPENING	True	Enable iterative deepening
<i>Move Generation</i>		
MIN_GROUP_SIZE	5	Minimum viable group size
MAX_GROUPS_PER_TURN	2	Max simultaneous group moves
MIN_SPLIT_SIZE	8	Minimum size before splitting
SPLIT_RATIOS	[1.0, 0.5]	Available split ratios
ATTACK_MIN_WIN_PROBABILITY	0.7	Minimum probability to attack
<i>Evaluation</i>		
EVAL_MATERIAL_WEIGHT	100	Weight per unit advantage
EVAL_IDEAL_GROUP_COUNT	2	Target number of groups
EVAL_CONCENTRATION_BONUS	100	Bonus for concentration
EVAL_FRAGMENTATION_PENALTY	250	Penalty per excess group
EVAL_SMALL_GROUP_THRESHOLD	10	Size for “small” group
EVAL_SMALL_GROUP_PENALTY	100	Penalty per small group
EVAL_RESOURCE_VALUE	40	Value per accessible human
EVAL_CENTER_CONTROL_WEIGHT	2	Weight for center control

B Battle Probability Table

Table 7: Win Probabilities for Different Force Ratios

Attackers : Defenders	Win Probability	Outcome Type
10 : 10	50.0%	Random
10 : 15	33.3%	Random
10 : 20	25.0%	Random
15 : 10	75.0%	Random
20 : 10	100.0%	Guaranteed (vs enemy)
10 : 10	50.0%	Guaranteed (vs humans)

C Code References

Key implementation files and their primary functions:

- `ai/alphabeta.py:130-170` — Main search loop
- `ai/alphabeta.py:206-270` — Alpha-Beta recursion
- `ai/evaluation.py:72-150` — Phase-based evaluation
- `ai/evaluation.py:426-465` — Fragmentation scoring
- `ai/move_generator.py:15-36` — Battle probability
- `ai/move_generator.py:189-268` — Move generation

- `ai/game_state.py:86-210` — State representation
- `ai/config.py` — All configuration parameters